

# ResonantOS SDK Reference Demo

Technical Report and User Guide

ResonantOS SDK Working Group

September 24, 2026

## Abstract

This report documents the ResonantOS SDK reference demonstration (SDK-DEMO-002), in which three independently packaged add-ons are discovered, validated, opened, and exercised inside the real ResonantOS browser extension through a single generic contract, with no add-on-specific code in ResonantOS Core. It describes the objectives, the architecture that was verified, the reference add-ons that were built, the capability and isolation model, the engineering problems that were found and corrected, the test results, and a step-by-step user guide for reproducing the demonstration.

## Contents

|          |                                                             |          |
|----------|-------------------------------------------------------------|----------|
| <b>1</b> | <b>Objectives</b>                                           | <b>2</b> |
| <b>2</b> | <b>Architecture Overview</b>                                | <b>2</b> |
| 2.1      | Isolation and token delivery . . . . .                      | 3        |
| <b>3</b> | <b>The Reference Add-ons</b>                                | <b>3</b> |
| 3.1      | Resonant Echo . . . . .                                     | 3        |
| 3.2      | Resonant Counter . . . . .                                  | 4        |
| 3.3      | SDK Guide . . . . .                                         | 4        |
| <b>4</b> | <b>Capability and Security Model</b>                        | <b>4</b> |
| <b>5</b> | <b>How It Was Built</b>                                     | <b>4</b> |
| 5.1      | Engineering problems found and corrected . . . . .          | 4        |
| 5.2      | Test hardening . . . . .                                    | 5        |
| <b>6</b> | <b>Test Results</b>                                         | <b>5</b> |
| <b>7</b> | <b>User Guide</b>                                           | <b>5</b> |
| 7.1      | Prerequisites . . . . .                                     | 5        |
| 7.2      | Checkout and install . . . . .                              | 6        |
| 7.3      | Place the add-ons where the bridge discovers them . . . . . | 6        |
| 7.4      | Start the bridge . . . . .                                  | 6        |
| 7.5      | Load the extension . . . . .                                | 6        |
| 7.6      | Exercise each add-on . . . . .                              | 6        |
| 7.7      | Run the test suite . . . . .                                | 7        |
| <b>8</b> | <b>Known Limitations and Next Steps</b>                     | <b>7</b> |

## 1 Objectives

The goal of this work was to move the ResonantOS SDK from an architectural proposal to a working, browser-visible demonstration. Specifically, the demonstration had to answer one question:

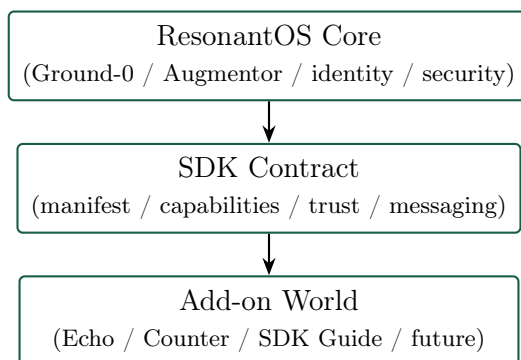
*Can a developer create a new, interactive ResonantOS add-on through a generic SDK contract, have ResonantOS discover it, expose its declared capabilities and trust boundary, open it inside the real browser extension, communicate with it, and disable or remove it — all without adding add-on-specific logic to ResonantOS Core?*

The acceptance criteria were intentionally strict. The demonstration had to run in the actual ResonantOS Chrome extension (not a mock page and not a terminal trace), and it had to prove the following properties:

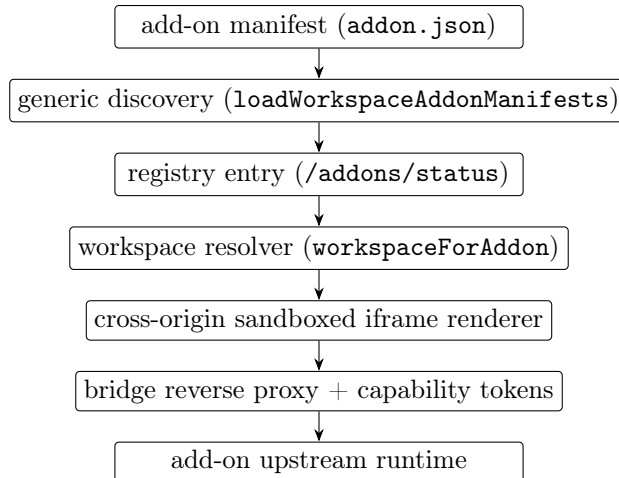
- **Generic discovery** — an independently packaged add-on is discovered without teaching Core its identifier.
- **Manifest-driven contract** — identity, version, capabilities, permissions, and contributions come from the add-on's own manifest.
- **Isolation** — add-on UI runs in a sandboxed, cross-origin context and cannot reach ResonantOS internals.
- **Capability enforcement** — an allowed operation succeeds; an unauthorized operation is refused at the boundary.
- **Replacement proof** — at least two unrelated add-ons register through the same mechanism, and a third, tutorial-style add-on teaches the SDK while itself running through that mechanism.

## 2 Architecture Overview

The SDK is a *plug/socket* contract, not a replacement for ResonantOS architecture. On one side is the trusted ResonantOS Core; on the other is the replaceable add-on world; between them is the SDK boundary through which every interaction must travel.



The end-to-end pipeline that a workspace add-on traverses is:



## 2.1 Isolation and token delivery

The key security decision is how add-on code executes. Add-ons render inside a sandboxed `iframe` whose `src` is the add-on's own upstream origin (for example `http://127.0.0.1:47321`), using the sandbox `allow-scripts` plus `allow-same-origin`. Because the `iframe` is same-origin with its own upstream and cross-origin from the extension, it can fetch its own API without CORS, yet cannot reach the extension's `chrome.*` APIs, extension storage, or the bridge.

Capability tokens are delivered from the trusted extension to the add-on `iframe` via `postMessage` (with `targetOrigin` set to the add-on's own origin). The add-on validates the message before trusting it. The bridge token is *never* delivered to the add-on, and no `Access-Control-Allow-Origin` header is used in the add-on upstreams.

## 3 The Reference Add-ons

Three add-ons were built, all registered through the same generic mechanism with zero Core changes:

| Add-on           | ID                                  | Upstream port | Purpose                                              |
|------------------|-------------------------------------|---------------|------------------------------------------------------|
| Resonant Echo    | <code>addon.resonant-echo</code>    | 47321         | Deterministic echo; proves the message round-trip    |
| Resonant Counter | <code>addon.resonant-counter</code> | 47422         | Second add-on; proves the mechanism is generic       |
| SDK Guide        | <code>addon.sdk-guide</code>        | 47423         | Interactive tutorial; teaches the SDK while using it |

### 3.1 Resonant Echo

A deterministic echo responder. A user types a message and the add-on returns **Resonant Echo received:** `"<message>"`. It has no model, no provider, and no network dependency. Its routes are `GET /api/echo/status` and `POST /api/echo/message`, both gated on the `harness-messaging` capability.

## 3.2 Resonant Counter

A deterministic counter. It exposes `POST /api/counter/increment` and `GET /api/counter/read`. Its existence is the *replacement proof*: a second, unrelated add-on onboards through the exact same mechanism with no additional Core code.

## 3.3 SDK Guide

A tutorial add-on that walks any user through what the SDK is, how an add-on is discovered and activated, what it can and cannot do, and what has been built. Its interactive steps perform *real* bridge-mediated calls: a live message round-trip and a deliberately denied action that returns 403. This is the strongest form of the replacement proof — an add-on that teaches the SDK while itself being an SDK add-on.

# 4 Capability and Security Model

Each add-on declares its capabilities in its manifest. The capability boundary is enforced at the bridge *and* re-checked at the add-on's own upstream.

| Status  | Capabilities                                                                                                                        |
|---------|-------------------------------------------------------------------------------------------------------------------------------------|
| Granted | <code>harness-messaging</code> (the single capability the reference add-ons use to send and receive messages)                       |
| Denied  | <code>wallet-signing</code> , <code>provider-secret-read</code> , <code>trusted-memory-write</code> , <code>filesystem-write</code> |

The security posture that was verified:

- The add-on iframe is isolated (cross-origin sandbox) and cannot reach extension storage, `chrome.*` APIs, or the bridge.
- Capability tokens are minted by the extension via the bridge and delivered to the iframe over `postMessage`; they are never placed in served HTML or a URL.
- The bridge token never reaches the add-on.
- No `Access-Control-Allow-Origin` header is emitted by the add-on upstreams.
- Unauthorized requests fail closed with 403.

# 5 How It Was Built

## 5.1 Engineering problems found and corrected

The independent review loop surfaced and fixed a series of real defects that the initial automated suite did not catch. These are recorded because they explain *why* the demonstration now works, and why a browser-visible test is a necessary gate:

1. **SyntaxError in the workspace module** — an `await` inside a non-`async` function broke the workspace page; the module was made `async` and a syntax gate was added to catch this class of error in CI.

2. **Inline script dropped by CSP** — the extension’s `script-src` policy silently dropped inline scripts in `srcdoc` iframes, leaving the add-on UI dead.
3. **Missing token delivery** — the bootstrap endpoint was capability-gated while nothing actually delivered the capability token to the iframe (a chicken-and-egg).
4. **Insecure CORS workaround** — an `Access-Control-Allow-Origin: null` plus HTML-templated tokens created a cross-origin read channel; this was replaced with `postMessage` delivery.
5. **Port not surfaced** — the registry entry derived the upstream port from an environment variable rather than the manifest’s `runtime.port`, so the renderer fell back to a non-isolated path and never delivered the bootstrap.
6. **Collapsed iframe** — the add-on iframe had no height propagation and rendered at the browser’s 150px default, clipping the input and Send button.

## 5.2 Test hardening

A dedicated unit test now pins the port-derivation logic (`buildWorkspaceAddonRegistryEntry`) so a regression of the `runtime.port` mapping is caught automatically. The real-extension browser test was changed to route its mock `/addons/status` response through the production derivation rather than hard-coding the port.

## 6 Test Results

| Suite                                      | Result                          |
|--------------------------------------------|---------------------------------|
| <code>npm run build</code>                 | PASS                            |
| <code>npm run test:extension-syntax</code> | 2/2 PASS                        |
| <code>npm run test:browser-host</code>     | 13/13 PASS                      |
| <code>npm run test:browser-first</code>    | 1298 PASS / 0 FAIL / 1 env-skip |
| <code>npm test (Vitest)</code>             | 487/487 PASS                    |

The single skip is environment-dependent: an in-process intake test skips when another bridge instance is already holding the add-on ports. On a clean machine the run is 0 skip. The real-extension browser tests additionally verify the visible click-through (Echo send, Counter increment, SDK Guide live message and denied action, and the unauthorized 403).

## 7 User Guide

This section describes how to reproduce the demonstration from a clean checkout.

### 7.1 Prerequisites

- Node.js `>= 22.13.0` (see `.nvmrc`).
- Google Chrome (for loading the unpacked extension).

## 7.2 Checkout and install

```
git clone https://github.com/vonstegen/2.0.0-alpha.git
cd 2.0.0-alpha
git checkout r-and-d/sdk-demo-002-cross-origin-addons
npm install
```

## 7.3 Place the add-ons where the bridge discovers them

The bridge reads workspace add-ons from the user root (`~/ResonantOS_User/BrowserFirst/addons`), not from the repository. Copy (do not symlink) the three add-on directories:

```
mkdir -p ~/ResonantOS_User/BrowserFirst/addons
cp -R browser-first/addons/resonant-echo ~/ResonantOS_User/BrowserFirst/addons/
cp -R browser-first/addons/resonant-counter ~/ResonantOS_User/BrowserFirst/addons/
cp -R browser-first/addons/sdk-guide ~/ResonantOS_User/BrowserFirst/addons/
```

## 7.4 Start the bridge

```
node run-bridge-minimal.mjs --bridge-port=47773
```

Leave the bridge running. It spawns the three add-on upstreams and writes the generated bridge configuration that the extension reads.

## 7.5 Load the extension

1. In Chrome, open `chrome://extensions`.
2. Enable **Developer mode**.
3. Choose **Load unpacked** and select `browser-first/resonantos-side-panel-extension`.
4. Open the ResonantOS side panel and navigate to **Add-ons**.

## 7.6 Exercise each add-on

1. **Resonant Echo** — click **Open Resonant Echo**, type `Hello Manolo`, and press **SEND**. The response `Resonant Echo received: "Hello Manolo"` renders, and the message counter increments.
2. **Resonant Counter** — click **Open Resonant Counter** and press the increment button; the counter advances.
3. **SDK Guide** — step through the tutorial. The live message step returns cross-boundary evidence, and the denied-action step shows a 403.

## 7.7 Run the test suite

```
npm run build
npm run test:extension-syntax
npm run test:browser-host
npm run test:browser-first
npm test
```

## 8 Known Limitations and Next Steps

- The work lives on the R&D branch `r-and-d/sdk-demo-002-cross-origin-addons`; it has not been merged into `dev` and remains subject to human review.
- The demonstration is intentionally deterministic. Replacing the deterministic responder with a real provider (the “Grok M1” step) is a separate milestone that should reuse the same socket.
- An n8n observatory workflow (SDK-DEMO-003) is planned as a read-only visualization layer. n8n will observe and display the SDK lifecycle; it will never be part of the authority path.
- A single environment-dependent test skip occurs when another bridge instance is holding the add-on ports; this is unrelated to the demo and disappears on a clean machine.

## 9 Conclusion

The ResonantOS SDK reference demonstration proves, in the real browser extension, that a third-party-style add-on can be discovered, validated, opened, and exercised through a single generic contract with no add-on-specific Core code. Three add-ons — Resonant Echo, Resonant Counter, and the SDK Guide — onboard through the same mechanism, capability enforcement fails closed, the add-on iframe is isolated, and the full automated suite is green. The result is a working extension boundary rather than a documented aspiration.